Au-delà de CGNet : exploration de stratégies d'échantillonnage et d'architectures pour la détection d'événements climatiques extrêmes

Alexis Lavigne **Rémi Lalonde** **Laurent Faucher**

Étienne Mitchell-Bouchard **Martin Medina Mejia**

Github du projet

Abstract

Que ce soient les rivières atmosphériques (AR) ou les cyclones tropicaux (TC), les événements climatiques extrêmes sont de plus en plus fréquents en raison du réchauffement climatique, d'où l'importance du développement de méthodes automatiques afin de les détecter. Ce projet se concentre sur la segmentation sémantique de TC et d'AR dans le jeu de données ClimateNet, qui possède un déséquilibre sévère de classes. En itérant sur l'architecture CGNet utilisée par Kapp-Schwoerer et al. (2020) et sur les travaux de Lacombe et al. (2023), deux approches ont été explorées : (1) des méthodes d'échantillonnage par sélection de fichiers et de patches afin de réduire le déséquilibre des classes, et (2) des architectures alternatives, notamment les U-Net et les ConvLSTM, pour mieux capturer la dimension temporelle des événements climatiques. La seconde approche a été plus efficace, surtout avec l'obtention des meilleures performances globales par l'Attention U-Net. Cette architecture a permis d'obtenir un IoU TC de 0.3765, soit une augmentation de 0.027 par rapport à la *baseline* sur cette classe, au prix d'un léger recul sur la segmentation des AR. Les résultats obtenus soulignent que l'enjeu principal reste le déséquilibre extrême des classes, particulièrement sur les TC, et que l'exploration d'architectures plus adaptées est plus prometteuse que les méthodes d'échantillonnage de données.

1 Introduction

Les événements météorologiques extrêmes ont des impacts majeurs et dévastateurs sur notre société, notre économie et nos infrastructures. De nos jours, avec le réchauffement climatique, la fréquence et l'intensité de ces phénomènes évoluent de manière imprévisible. C'est pourquoi leur détection automatique et précise à partir de données atmosphériques est devenue cruciale pour la gestion des risques.

Ce projet se concentre plus particulièrement sur l'identification de deux types de phénomènes : les cyclones tropicaux (TC), les événements météorologiques les plus destructeurs au monde, caractérisés par des vents à très grande vitesse avec une forte rotation locale, et les rivières atmosphériques (AR), d'immenses corridors d'humidité dans l'atmosphère qui transportent de grandes quantités de vapeur d'eau et sont responsables de précipitations extrêmes et d'inondations.

L'apprentissage profond a récemment prouvé que la segmentation d'images a un énorme potentiel pour remplacer les méthodes de détection traditionnelles, qui sont faites manuellement par des experts. En effet, Kapp-Schwoerer et al. (2020) ont entraîné un CGNet, une architecture CNN légère, sur ClimateNet, établissant la *baseline* pour cette tâche, que Lacombe et al. (2023) ont ensuite approfondie. Malgré ces travaux, certains défis sont toujours présents. Tout d'abord, le jeu de données

présente un fort déséquilibre des classes, ce qui pousse les modèles à ignorer les événements rares. De plus, les données climatiques à disposition sont extrêmement lourdes. Cela rend les entraînements lents et très exigeants en mémoire. Puis, la majorité des modèles existants analysent le climat comme de simples images statiques indépendantes, en omettant l'évolution temporelle des événements.

Ce projet explore deux voies pour répondre à ces limites: des méthodes d'échantillonnage de fichiers et de patches pour réduire le déséquilibre des classes et la taille des données, ainsi que des architectures alternatives, comme les U-Net et les ConvLSTM, pour améliorer les performances et mieux capturer la dimension temporelle.

2 Revue de littérature

Historiquement, la détection d'événements climatiques extrêmes reposait sur des heuristiques manuelles et spécifiques aux types d'événements. Cette approche souffrait d'un manque de cohérence et d'ambiguïté entre les chercheurs [1]. Prabhat et al. (2020) ont proposé une approche d'apprentissage profond supervisé en créant *ClimateNet*, un jeu de données annoté par des experts, et en appliquant *DeepLabv3+* [2] à la segmentation sémantique des rivières atmosphériques (AR) et cyclones tropicaux (TC). Le modèle atteint un *Intersection over Union* (IoU) de 52,47 % sur *ClimateNet*, ce qui est comparable à l'accord moyen entre les experts (51,20 %), et suggère qu'un IoU parfait est inaccessible sur ces données [3].

Kapp-Schwoerer et al. (2020) ont privilégié l'efficacité computationnelle avec une architecture *CGNet* [4], 80 fois plus léger que *DeepLabv3+*, atteignant un IoU de 59,3 % sur l'ensemble d'entraînement et 56,1 % sur l'ensemble de test sur quatre variables climatiques (TMQ, U850, V850 et PSL) sélectionnées avec des experts climatiques. Ce modèle représente notre *baseline*. Cependant, leur réseau traite chaque image indépendamment, sans aspect temporel, ce qui nécessite un algorithme de post-traitement.

Cette limitation n'est pas adressée par Lacombe et al. (2023), bien qu'ils essayent plusieurs fonctions de perte pondérées, pour combattre le déséquilibre sévère entre les classes, l'ingénierie des caractéristiques et d'autres stratégies de contrôle. Ils tentent de maximiser le rappel pour minimiser les faux négatifs [5]. Leur meilleure expérience atteint un rappel TC de 79,44 % contre 57,08 % du baseline.

3 Jeu de données

Le jeu de données ClimateNet [3] contient des données du modèle climatique CAM5.1 annotées manuellement par des experts du domaine. Il est composé de 459 fichiers de format NetCDF ($\sim$30 Go), où chaque fichier représente un snapshot global de l'atmosphère sur une grille de 768×1152 pixels (latitude $\times$ longitude) à une résolution de 25 km. Chaque fichier est composé de 17 canaux, soit 16 variables atmosphériques et une étiquette, assignée pixel par pixel selon trois classes : *background* (93.8%), rivières atmosphériques (5.67%) et cyclones tropicaux (0.46%). Il est également séparé temporellement en entraînement (1996-2010) et test (2011-2013). Une particularité importante est que les fichiers ne sont pas tous consécutifs temporellement. En effet, certains jours contiennent plusieurs fichiers alors que d'autres sont absents, créant des écarts qui peuvent atteindre 253 jours entre deux fichiers.

4 Méthodes

4.1 Optimisation des données

Le jeu de données ClimateNet contient presque 30 GB de données et présente un fort déséquilibre de classes (93.8% *background* (BG)), rendant l'apprentissage des événements extrêmes difficile. Afin de réduire ce déséquilibre et de réduire la quantité de données, deux approches de sous-échantillonnage ont été explorées.

4.1.1 Échantillonnage par sélection de fichiers

La première approche consiste simplement à sélectionner les fichiers contenant les plus grandes proportions de TC et d'AR selon un score pondéré :

$$s = \alpha \times \%_{\mathrm{TC}} + \beta \times \%_{\mathrm{AR}} \tag{1}$$

où $\alpha = 12$ et $\beta = 1$ reflètent le déséquilibre des classes. Cela permet de traiter les TC et les AR de manière équivalente lors de la sélection de fichiers afin que les TC ne soient pas désavantagés, car il y a 12 fois plus de pixels AR que de pixels TC dans le jeu de données ClimateNet. Par ailleurs, lors de l'exploration des données, nous avons réalisé que 27% des pixels ne reçoivent jamais l'étiquette TC ou AR. Un `safe_mask` a donc été intégré dans la perte Jaccard afin d'ignorer les zones géographiquement inactives. Ainsi, même si l'échantillonnage par fichiers réduit significativement la taille du jeu de données, il n'améliore pas vraiment le débalancement entre les classes. C'est pour cela que l'approche par patches a été explorée aussi.

4.1.2 Échantillonnage par patches

Au lieu d'ignorer des fichiers complets, la seconde approche échantillonne des patches de 128×128 pixels sur tous les fichiers de taille 768×1152. Cela divise donc chaque fichier en 54 patches: certaines ayant des pixels TC, certaines ayant des pixels AR et certaines n'ayant que des pixels BG. Le paramètre *BG_ratio* permet de choisir quel pourcentage de patchs entièrement BG seront gardés lors de l'entraînement. Cette approche s'attaque non seulement à la réduction de données, mais aussi au déséquilibre des classes puisqu'elle réduit agressivement de 80% le taux de pixels BG lors de l'entraînement. La figure 1 illustre l'échantillonnage par patches sur un seul fichier. De plus, cette méthode a aussi permis de tester un suréchantillonnage des patches TC, la classe la plus sous-représentée, en réintroduisant les patches par augmentation de données avec les rotations suivantes: $90°$, $180°$, $270°$.

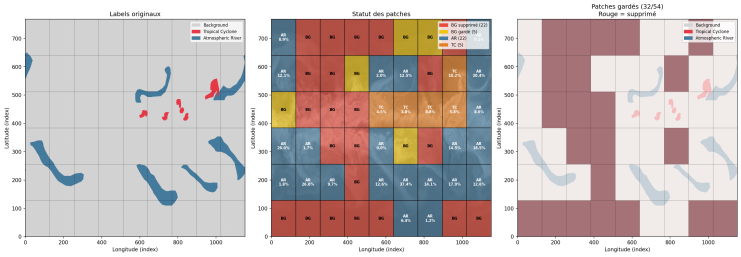


Figure 1: Visualisation des patches: fichier 2010-10-29-01-1_0.nc. Patch 128 x 128, BG_Ratio = 20%, 32/54 patches gardés.

Puisque CGNet est entraîné sur des patches de 128×128, une procédure d'évaluation sur la grille complète 768×1152 à été implémentée. Cette méthode consiste à faire glisser une fenêtre de 128×128 sur la grille complète avec un stride de 64 pixels, ce qui engendre un chevauchement de 50% entre les patches. Chaque pixel peut donc être couvert par jusqu'à 4 patches et donc jusqu'à 4 prédictions. La classe finale du pixel est déterminée par la moyenne des probabilités *softmax* des patches le couvrant, suivie d'un *argmax* sur les 3 classes (BG, TC, AR). Cette méthode de *sliding window* est moins efficace à l'inférence, mais elle assure une prédiction cohérente pour les TC et AR qui s'étendent sur plus d'un patch.

4.2 Exploration d'architectures différentes à CGNet

En parallèle des stratégies d'échantillonnage, des architectures différentes du CGNet original ont été testées afin d'évaluer si un changement de structure pouvait mieux répondre à la tâche de segmentation.

4.2.1 U-Net

L'architecture U-Net classique [6] a été implémentée comme premier point de comparaison. Ce modèle repose sur une structure encodeur-décodeur standard : l'encodeur applique une série de doubles convolutions suivies d'un sous-échantillonnage par max pooling pour extraire les caractéristiques spatiales, tandis que le décodeur utilise des convolutions transposées pour restaurer la résolution spatiale. Ces données sont ensuite concaténées avec les cartes de caractéristiques de l'encodeur via des *skip connections*. Le modèle a été entraîné en utilisant une fonction de perte cross-entropy. Notre équipe a décidé d'expérimenter avec cette architecture, puisqu'il s'agit d'une référence en segmentation d'images, nous permettant donc d'avoir une base de comparaison robuste.

4.2.2 Attention U-Net

Configuration de Base : Afin de surmonter les limites rencontrées par le modèle U-Net standard, tel que sa tendance à générer de nombreux faux positifs, nous avons introduit un mécanisme d'attention. Ce mécanisme utilise les caractéristiques profondes du réseau pour évaluer l'importance des informations provenant des *skip connections*. Il génère une carte d'attention avec des valeurs comprises entre 0 et 1, où les valeurs élevées correspondent davantage aux régions pertinentes, comme les évènements climatiques, tandis que les valeurs faibles correspondent plutôt au fond de l'image. Cette carte est ensuite appliqué aux *skip connections* et permet de réduire l'influence des régions moins importantes.

Configuration améliorée : Pour améliorer les performances, plusieurs modifications ont été apportées à l'architecture de base et à la boucle d'entraînement. Étant limités par la mémoire du GPU (erreurs *Out of Memory* avec de grands batchs), une taille de batch physique de 2 a été maintenue, mais en accumulant les gradients sur 4 étapes. Cela équivaut donc à simuler une taille de batch de 8, ce qui a aidé à réduire le bruit lors de l'entraînement du modèle et à stabiliser l'apprentissage, au prix d'une augmentation du nombre d'époques de 25 et 30. En ce qui concerne la fonction de perte, la CrossEntropy a été combinée à la Dice Loss. La première est adaptée à la classification pixel par pixel et assure la stabilité de l'apprentissage global, tandis que la seconde optimise directement l'IoU et est mieux adaptée aux données déséquilibrées [7]. Des connexions résiduelles ont également été ajoutées dans les blocs de convolution de l'encodeur pour permettre au gradient de circuler plus facilement dans les couches profondes. Cela permet au modèle d'extraire des caractéristiques plus complexes sans souffrir de la disparition du gradient. [8]

Finalement, les modèles ont été entraînés sur l'ensemble de données contenant l'ingénierie des caractéristiques de Lacombe et al. (2023) afin d'essayer de permettre au réseau de mieux segmenter les frontières des cyclones et des rivières atmosphériques par rapport au fond, et exploré l'ajout d'une couche de dropout au niveau du *bottleneck* pour prévenir le surapprentissage.

4.2.3 ConvLSTM hybrides

Les modèles comme CGNet et UNet traitent chaque fichier climatique de manière indépendante, en ignorant la dimension temporelle des événements climatiques extrêmes. Afin de pallier à cette limitation, des architectures ConvLSTM [9] ont été explorées afin d'intégrer la continuité temporelle directement dans la segmentation, sans nécessiter d'algorithme de post-traitement.

En raison des contraintes de mémoire des GPU disponibles, des architectures hybrides ont été implémentées. D'abord, le `ConvLSTM-UNet` combine un encodeur ConvLSTM à quatre niveaux avec un décodeur UNet qui possède des *skip connections*. Ensuite, le `ConvLSTM-UNet-GC` est une variante avec l'ajout d'un module de contexte global au niveau du goulot d'étranglement, inspiré du CGBlock de Kapp-Schwoerer et al. [1]. D'autres architectures comme un décodeur ConvLSTM complet (`ConvLSTM-UNet-FullLSTM`) et un ConvLSTM pur sans décodeur ont été testées, mais étaient limitées par des erreurs de mémoire (*OOM*).

En raison de la présence d'écarts temporels importants entre certains fichiers de données (plus grand écart de 253 jours), deux approches ont tenté de garantir une meilleure continuité temporelle avant l'entraînement. La première considère uniquement les fichiers séquentiels du même jour, et la deuxième vient remplacer les écarts par des trames de fond (de classe BG).

Plusieurs ablations ont également été effectuées sur la longueur de séquence T, le nombre de caractéristiques utilisées, les poids des fonctions de perte pondérées, la fonction de perte, le *dropout* et le *scheduler* du taux d'apprentissage.

4.3 Exploitation du travail futur de l'article de 2020 de Kapp-Schwoerer et al.

En complément de ces architectures alternatives, nous avons aussi étudié plusieurs pistes d'amélioration directement liées au CGNet original, inspirées des travaux futurs proposés par Kapp-Schwoerer et al. [1]. Plus précisément, trois directions ont été explorées : l'impact du nombre de canaux d'entrée, la quantification d'incertitude par MC Dropout, ainsi que l'intégration de la temporalité via une architecture CGNet + ConvLSTM.

En ce qui concerne le premier axe, le CGNet original utilisant 4 canaux a été comparé à des variantes utilisant 8 et 20 canaux, avec ou sans ajout de caractéristiques engineered, afin d'évaluer l'effet d'une représentation d'entrée plus riche sur les performances du modèle.

Le second axe portait sur la quantification d'incertitude à l'aide du MC Dropout. Cette approche consiste à conserver le dropout actif lors de l'inférence et à effectuer plusieurs passes stochastiques sur une même entrée. Dans nos expériences, 30 passes ont été réalisées. La moyenne des sorties softmax a été utilisée comme prédiction finale, tandis que leur variance a servi d'estimation de l'incertitude pixel par pixel.

Enfin, le troisième axe visait à intégrer la dimension temporelle du problème. Pour cela, une architecture hybride combinant un CGNet préentraîné et un ConvLSTM appliqué sur les feature maps a été étudié. Deux paramètres ont été analysés: la longueur de séquence T et la dimension cachée h. Les configurations testées incluent notamment ($T = 3$, $h = 64$), ($T = 3$, $h = 128$) et ($T = 5$, $h = 128$).

Comme les expériences temporelles reposent sur seulement 92 séquences consécutives, les résultats de l'architecture hybride doivent être interprétés comme exploratoires.

5 Résultats

5.1 Méthodes d'échantillonage

Le tableau 1 présente les résultats de l'échantillonnage par fichiers avec l'équation (1). On remarque que les résultats présentés sont légèrement inférieurs à la baseline [1]. L'ajout du *safe_mask* semble uniquement améliorer le rappel au détriment des autres métriques de performance. La configuration optimale retenue est la sélection des 75% des fichiers les mieux classés. En effet, l'expérience 2 offre des résultats supérieurs pour la détection de AR mais inférieurs pour la détection de TC ce qui est encourageant considérant une réduction de 25% des données. **N. B. :** Dans le tableau 1, 64% a été choisi comme seuil pour conserver au moins 90% des pixels TC dans tous les fichiers. À un seuil de 75%, 95,4% des pixels TC sont conservés.

Table 1: Comparaison des expériences de sélection de fichiers avec et sans *safe_mask*. Les valeurs en **gras** indiquent le meilleur résultat par colonne.

#	Expérience	Cyclone tropical (TC)				Rivière atmosphérique (AR)			
		IoU	Préc.	Rap.	Spéc.	IoU	Préc.	Rap.	Spéc.
0	Reproduction baseline [1]	**0.3486**	**0.4599**	0.5903	**0.9961**	0.3848	0.4543	0.7157	0.9491
1	Top 64% fichiers	0.3359	0.4345	0.5970	0.9957	0.3807	0.4502	0.7114	0.9485
2	Top 75% fichiers	0.3304	0.4391	0.5718	0.9960	**0.3914**	**0.4973**	0.6475	**0.9612**
3	Top 64% + safe_mask	0.3193	0.3812	**0.6628**	0.9941	0.3648	0.4250	**0.7200**	0.9423
4	Top 75% + safe_mask	0.3220	0.4061	0.6084	0.9951	0.3910	0.4900	0.6595	0.9593

La Table 2 présente les meilleurs résultats de l'échantillonnage par patches 128×128 (la table complète se trouve à l'Annexe A). Afin de trouver le *bg_ratio* optimal, nous avons diminué progressivement ce paramètre de 40% à 0%. L'expérience 3 avec un *bg_ratio* de 20% présente le meilleur compromis parmi ces configurations. Ensuite, nous avons essayé d'augmenter les données en réintroduisant les patches contenant des pixels de TC avec des rotations aléatoires ($90°$, $180°$, $270°$). Contre-intuitivement, les augmentations effectuées aux expériences 8 et 9 dans la table 7 ont uniquement amélioré les performances de détection des AR et n'ont donc pas été retenues. Finalement, nous avons combiné l'échantillonnage par patch avec des fonctions de pertes pondérées aux expériences 10 et 11. Ces deux expériences ont uniquement augmenté le rappel en réduisant significativement toutes les autres métriques de performance et n'ont donc pas été retenues non plus.

Le tableau 8 en Annexe A résume la réduction des données obtenue par les deux techniques d'échantillonnage pour 4 et 16 caractéristiques. La sélection des 75% des fichiers les mieux classés réduit le jeu de données de 9,75 Go à 7,31 Go (25%) en utilisant les 4 caractéristiques de Kapp-Schwoerer et al. [1]. Quant à la méthode par patch, elle réduit davantage le jeu de données de 55%, soit de 9,75 à 4,35 Go pour 4 caractéristiques, et de 32,3% (29,26 Go à 19,80 Go) pour les 16 carac-

Table 2: Comparaison des expériences d'échantillonnage par patches. $\star$ configuration retenue. Les valeurs en **gras** indiquent le meilleur résultat par colonne.

#	Expérience	Cyclone tropical (TC)				Rivière atmosphérique (AR)			
		IoU	Préc.	Rap.	Spéc.	IoU	Préc.	Rap.	Spéc.
0	Reproduction baseline [1]	**0.3486**	0.4599	0.5903	0.9961	0.3848	0.4543	0.7157	0.9491
2	BG 15%	0.3005	0.5150	0.4191	**0.9978**	0.3821	0.5464	0.5597	0.9725
3	BG 20% $\star$	0.3126	0.4538	0.5012	0.9967	0.3854	0.5065	0.6172	0.9644
7	BG 20% + all features	0.3321	**0.5211**	0.4781	0.9976	**0.3893**	0.5242	0.6022	0.9676
8	BG 20% + TC $\times 6$	0.3071	0.4750	0.4650	0.9972	0.3764	**0.5513**	0.5427	**0.9738**
11	BG 20% + Jaccard pondérée	0.2469	0.2731	**0.7207**	0.9894	0.3588	0.4179	**0.7173**	0.9408

téristiques. L'utilisation de petits patches 128×128 permet également de stabiliser l'entraînement en augmentant le batch size de 2 à 32, les ressources mémoire du cluster HPC étant limitantes sans cette approche. En somme, ces deux méthodes permettent d'obtenir des résultats similaires à la baseline [1] tout en réduisant significativement la taille de ClimateNet d'au moins 25%.

5.2 U-Net

La première ligne de la Table 3 présente les résultats du U-Net standard et montre qu'il s'agit d'un modèle très sensible. Ses excellents scores de rappel en test (0.8702 pour les TC et 0.9151 pour les AR) assurent la détection de presque la totalité des événements. Cependant, cette sensibilité introduit de nombreux faux positifs, ce qui fait que les résultats de la précision sont très mauvais (0.1433 pour les TC, 0.2321 pour les AR) et pénalise aussi l'IoU global. Malgré cela, la grande similarité des métriques d'entraînement et de test confirme la stabilité du modèle et l'absence de surapprentissage.

5.3 Attention U-Net

Table 3: Comparaison des performances des différentes architectures et configurations du U-Net standard et Attention U-Net sur l'ensemble de test. Les valeurs en **gras** indiquent le meilleur résultat par colonne.

#	Configuration	Cyclone tropical (TC)				Rivière atmosphérique (AR)			
		IoU	Préc.	Rap.	Spéc.	IoU	Préc.	Rap.	Spéc.
1	U-Net standard	0.1496	0.1433	**0.8865**	-	0.2273	0.2321	**0.9151**	-
2	Base Attention U-Net (15 époques)	0.3495	0.4256	0.6617	0.9951	**0.4014**	**0.5317**	0.6208	**0.9676**
3	Base + Feature Eng. (15 époques)	0.3283	0.3848	0.6909	0.9939	0.3767	0.4701	0.6547	0.9563
4	Amélioré + Feature Eng. (25 époques)	**0.3765**	**0.5216**	0.5751	**0.9971**	0.3879	0.4701	0.6893	0.9540
5	Amélioré + Feature Eng. + dropout (25 époques)	0.3536	0.4534	0.6164	0.9959	0.3813	0.4551	0.7018	0.9502
6	Amélioré + Feature Eng. (30 époques)	0.3462	0.4104	0.6888	0.9945	0.3846	0.4604	0.7004	0.9514

En comparant les résultats dans la Table 3, on remarque que les résultats du U-Net avec mécanisme d'attention sont nettement meilleurs que ceux du modèle standard. De même, on remarque que le modèle avec configuration améliorée, sans dropout et entraîné sur 25 époques, est celui qui donne le meilleur compromis global. Bien que le modèle avec la configuration de base ait eu une excellente performance sur les rivières atmosphériques (AR), la version améliorée sans dropout (25 époques) offre un meilleur équilibre en augmentant la précision et l'IoU des cyclones tropicaux (TC). La table 3 confirme également que le dropout n'est pas une bonne décision, puisqu'il n'améliore pas vraiment les résultats et en plus, sans celui-ci, il n'y a pas de signes de surapprentissage sur 25 époques.

En contrepartie, on remarque que le prolongement de l'entraînement à 30 époques fait diminuer l'IoU et la précision des cyclones, signifiant que le modèle génère trop de faux positifs et souffre de surapprentissage. Afin de confirmer cela, nous avons analysé les courbes d'entraînement du modèle amélioré sans dropout entraîné sur 30 époques, disponibles dans la Figure 3 en Annexe B. Bien que la fonction de perte (Figure 3a) démontre une convergence stable, l'IoU sur l'ensemble de test (Figure 3b) atteint sa valeur maximale entre les époques 20 et 25 avant de commencer à surapprendre légèrement. De plus, les métriques de rappel et de précision (Figures 3c et 3d) sont très bruités, probablement à cause de la petite taille de batch, sur laquelle nous sommes limités.

En conclusion, ces résultats montrent que la configuration qui donne le meilleur compromis, tout en évitant le surapprentissage, est notre version améliorée sans dropout entraînée sur 25 époques. Cela dit, malgré les bons résultats, le réseau est encore très sensible aux variations locales de chaque batch, ce qui ne permet pas d'avoir des métriques d'entraînement stables et cause une forte volatilité.

5.4 ConvLSTM hybrides

Bien que l'Attention U-Net améliore les performances de segmentation spatiale, il traite encore chaque image indépendamment. Les expériences suivantes explorent donc l'ajout d'une composante temporelle à l'aide de ConvLSTM.

Les résultats des expériences les plus pertinentes sont présentés dans la Table 9 (la Table complète se trouve en Annexe C). Plusieurs observations ressortent des expériences effectuées. Premièrement, une longueur de séquence $T = 3$ (expérience 2) présente le meilleur compromis pour les deux classes, bien que $T = 2$ obtienne des résultats légèrement supérieurs pour les TC. $T = 4$ présente une grande baisse d'IoU pour les TC. Deuxièmement, l'utilisation de 16 caractéristiques du jeu de données avec l'architecture `ConvLSTM-UNet-GC` améliore les deux IoU ainsi que le rappel AR, contrairement aux quatre variables initiales. Troisièmement, les deux stratégies qui visaient à améliorer la cohérence temporelle des séquences n'ont pas été bénéfiques. En effet, en considérant uniquement les fichiers séquentiels d'un même jour (expérience 8), le nombre de séquences d'entraînement est réduit d'un facteur d'environ 3.3, ce qui représente une perte trop importante de données. De plus, même en conservant l'entièreté des données et en garantissant la continuité temporelle en remplissant les écarts temporels par trames BG (expérience 12), cela revient à sur-échantillonner la classe majoritaire et vient dégrader les performances de détection, notamment l'IoU et le rappel. Finalement, des techniques comme le *dropout* ou l'utilisation d'un *scheduler* cosinus n'ont pas amélioré les résultats.

Table 4: Comparaison des expériences ConvLSTM sur l'ensemble de test (époque 15). Sauf indication contraire : perte d'entropie croisée pondérée $[0.5, 5.0, 2.0]$, caractéristiques TMQ, U850, V850, PSL [1], séquences consécutives, `features=[8, 16, 32, 64]` pour ConvLSTM-UNet, et `hidden_channels=[8, 16, 8]` pour ConvLSTM pur. T = longueur de séquence. †Caractéristiques : TMQ, WS850, VRT850, PSL, PRECT [5]. *OOM: erreur due au manque de mémoire. Les valeurs en **gras** indiquent le meilleur résultat par colonne.

#	Configuration	Cyclone tropical (TC)				Rivière atmosphérique (AR)			
		IoU	Préc.	Rap.	Spéc.	IoU	Préc.	Rap.	Spéc.
0	Reproduction baseline [1]	0.3486	0.4599	0.5903	0.9961	0.3848	0.4543	0.7157	0.9491
ConvLSTM-UNet									
1	ConvLSTM-UNet, $T = 2$	0.3569	0.4998	0.5552	0.9969	0.3551	0.5965	0.4673	0.9813
2	ConvLSTM-UNet, $T = 3$	0.3495	0.4870	0.5531	0.9967	0.3786	0.5235	0.5777	0.9689
3	ConvLSTM-UNet, $T = 4$	0.2433	0.2587	**0.8034**	0.9869	0.3771	0.5928	0.5090	0.9794
5	#2 avec poids TC=7.0 dans la perte	**0.3699**	0.5381	0.5421	0.9974	0.3186	0.6452	0.3863	0.9874
7	#2 avec perte de Jaccard	0.2811	**0.6351**	0.3352	**0.9989**	0.3454	0.5921	0.4532	0.9815
8	#2 avec séquences du même jour seulement	0.2096	0.3313	0.3634	0.9957	0.3807	0.5019	0.6119	0.9609
ConvLSTM-UNet-GC									
11	ConvLSTM-UNet-GC, $T = 3$	0.3381	0.4329	0.6069	0.9984	0.3831	0.5389	0.5698	0.9711
12	#11 avec ajout de trames *BG* pour continuité	0.1716	0.5704	0.1970	0.9992	0.2343	0.3913	0.3687	0.9661
14	#11 avec 16 caractéristiques	0.3527	0.5015	0.5431	0.9970	**0.3856**	0.4961	0.6338	0.9619
15	#14 avec poids [0.36, 30.0, 5.88]	0.2557	0.2768	0.7700	0.9887	0.3474	0.3999	**0.7260**	0.9355
*Autres architectures limitées par *OOM*									
18	ConvLSTM-UNet (décodeur LSTM), $T = 2$	0.3377	0.4905	0.5201	0.9970	0.2500	**0.6916**	0.2814	**0.9926**

Globalement, la meilleure configuration est l'expérience **14** (voir Figure 4 en Annexe C), soit le `ConvLSTM-UNet-GC` avec les 16 caractéristiques et $T = 3$. Ce modèle atteint l'IoU AR le plus élevé parmi toutes les expériences (0.3856) et surpasse six des huit métriques du *baseline*. Cependant, il ne dépasse pas les rappels TC et AR du *baseline*, ce qui indique qu'il priorise la précision au lieu du rappel. Ces résultats confirment que l'intégration du contexte temporel avec des architectures ConvLSTM est pertinente pour la détection des événements climatiques extrêmes. Cependant, il y a deux limitations importantes : les écarts temporels irréguliers entre les fichiers, qui nuisent à la continuité temporelle, et les contraintes de mémoire qui limitent la taille des architectures.

5.5 Exploitation du travail futur de l'aticle de 2020 de Kapp-Schwoerer et al.

Après l'analyse des architectures alternatives, cette section revient sur des modifications ciblées du CGNet original. Le Tableau 5 montre que l'ajout de canaux d'entrée n'améliore pas clairement les performances, surtout pour les TC. Les variables supplémentaires semblent donc apporter peu d'information discriminante et peuvent même introduire de la redondance ou du bruit. La configuration originale à 4 canaux demeure ainsi la plus pertinente.

Table 5: Impact du nombre de canaux et de MC Dropout sur CGNet (époque 15). Les valeurs en **gras** indiquent le meilleur résultat par colonne.

Architecture	**Cyclone tropical (TC)**				**Rivière atmosphérique (AR)**			
	IoU	Préc.	Rap.	Spéc.	IoU	Préc.	Rap.	Spéc.
Reproduction baseline [1]	**0.3486**	0.4599	**0.5903**	0.9961	0.3848	0.4543	**0.7157**	0.9491
CGNet 8ch	0.3484	0.4750	0.5665	0.9965	**0.4046**	0.5065	0.6677	**0.9615**
CGNet 8ch + feat. ing. [5]	0.3468	**0.5289**	0.5018	**0.9975**	0.3991	0.4858	0.6910	0.9567
CGNet 20ch	0.3382	0.4511	0.5748	0.9961	0.3995	**0.5536**	0.5894	0.9609
CGNet + MC Dropout ($N = 30$)	**0.3514**	0.4579	**0.6017**	0.9961	**0.4054**	**0.5048**	0.6732	**0.9609**

Les résultats avec MC Dropout sont plus encourageants. Cette approche améliore légèrement l'IoU des TC sans dégrader fortement les AR. La moyenne de plusieurs prédictions rend la sortie plus stable, et l'incertitude se concentre surtout aux frontières des événements, comme montré à la Figure 5. Même si le gain reste limité, le MC Dropout apporte donc une information utile sur la fiabilité locale des prédictions.

L'intégration de la temporalité avec CGNet + ConvLSTM donne des résultats plus contrastés. Le Tableau 6 montre qu'augmenter la dimension cachée de h = 64 à h = 128 améliore légèrement les performances TC, mais qu'une séquence plus longue, T = 5, dégrade fortement les résultats. Cela s'explique probablement par le nombre réduit de séquences valides, qui pénalise surtout les TC, la classe la plus rare. Globalement, l'ajout de canaux n'améliore pas le CGNet, le MC Dropout apporte un léger gain de robustesse, et la modélisation temporelle demeure limitée par la faible quantité de séquences disponibles.

Table 6: Résultats du modèle hybride CGNet + ConvLSTM (époque 15, dataset engineered).

Architecture	**Cyclone tropical (TC)**		**Rivière atmosphérique (AR)**	
	IoU	Rap.	IoU	Rap.
Hybride $T = 3$, $h = 64$	0.193	0.618	0.360	**0.841**
Hybride $T = 3$, $h = 128$	**0.209**	0.614	0.353	**0.841**
Hybride $T = 5$, $h = 128$	0.111	0.282	**0.445**	0.809

5.6 Post-traitement

Pour suivre les événements à travers le temps, il est pertinent d'appliquer un système de post-traitement, comme mentionné dans [3]. Pour ce faire, lorsqu'un événement dépasse une aire minimale de 250 pixels et persiste au moins 12 heures, un identifiant lui est assigné. Il est ensuite suivi dans les prédictions suivantes à partir du chevauchement spatial entre les régions détectées.

Cette approche a d'abord été testée sur les données d'évaluation de ClimateNet. Cependant, comme les fichiers sont peu nombreux et discontinus dans le temps, les animations obtenues ne permettaient pas de bien évaluer le suivi temporel. Pour remédier à cette limite, nous avons aussi exploré ERA5, un ensemble de données plus récent et continu temporellement. Comme ERA5 ne contient pas exactement les mêmes caractéristiques que ClimateNet avec l'ingénierie de caractéristiques, nous avons entraîné l'Attention U-Net avec différents ensembles de caractéristiques afin d'identifier une configuration plus facilement transférable. Les résultats, présentés dans la table 10, montrent que les expériences 1 et 2 offrent les meilleures performances. Nous avons donc retenu le modèle entraîné

uniquement avec les quatre caractéristiques de CGNet, puisque ces variables sont plus simples à extraire dans ERA5. Le système a ensuite été testé sur les données d'évaluation de ClimateNet ainsi que sur la saison des ouragans de 2022 disponible dans ERA5. Les animations générées, accessibles sous forme de GIFs dans le dépôt GitHub et illustrées aux figures 6 et 7, montrent qualitativement que le modèle peut suivre certains événements. Une analyse plus ciblée sur l'ouragan Erin (figure 8) montre également que le modèle arrive à suivre sa progression dans la région des Caraïbes.

6 Discussion

La Figure 2 présente le compromis entre l'IoU et le rappel. Contrairement à Kapp-Schwoerer et al. (2020) qui évaluent leur modèle uniquement sur le IoU, nous pensons, comme Lacombe et al. (2023) qu'il est aussi pertinent de considérer le rappel. En effet, cette métrique est cruciale pour la détection d'événements climatiques extrêmes puisqu'un rappel élevé minimise les faux négatifs et un TC non détecté peut coûter des vies. Dans ce contexte risqué, il vaut mieux déclencher une alerte inutile (faux positif) que de manquer un événement réel (faux négatif). C'est donc pour cela que nous considérons la combinaison de ces deux métriques.

Sur la Figure 2 les points ronds représentent les résultats obtenus pour les TC tandis que les croix représentent les résultats obtenus pour les AR. La baseline [1] ainsi que notre modèle le plus performant (Attention U-Net) sont encerclés en bleu et en mauve respectivement afin d'être mis en évidence.

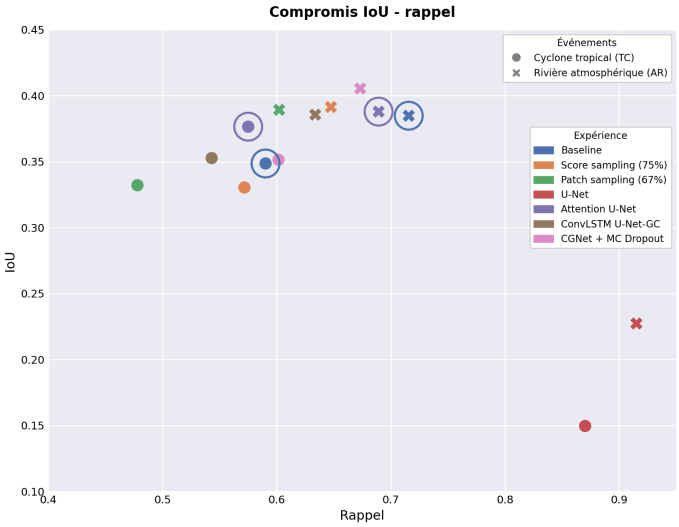


Figure 2: Comparaison de l'ensemble des résultats: les deux stratégies d'échantillonage et les architectures U-Net, Attention U-Net, ConvLSTM-U-Net-GC, et CGNet + MC Dropout.

La Figure 2 révèle clairement que les deux **méthodes d'échantillonnage** (vert et orange) obtiennent un IoU très proche de la baseline [1], mais un rappel légèrement plus faible. Ce léger recul est attendu considérant que ces approches réduisent le jeu de données d'entraînement de 25% à 33%. Quant à lui, le **ConvLSTM U-Net-GC** (brun) affiche des performances similaires aux méthodes d'échantillonnage, avec un IoU légèrement supérieur pour les TC. Cela suggère que les architectures incorporant une dimension temporelle n'améliorent pas significativement les résultats, possiblement en raison du manque de continuité temporelle entre les snapshots de ClimateNet. En ce qui concerne le **U-Net standard** (rouge), cette architecture affiche les résultats les plus extrêmes avec un IoU très faible, mais un rappel très élevé. Ceci suggère que ce modèle a tendance à surestimer l'occurrence de TC et d'AR au détriment significatif de la précision.

Comparaison des deux meilleures architectures : Le **CGNet avec MC Dropout** (rose) et l'**Attention U-Net** (mauve) sont les deux approches les plus intéressantes. Le MC Dropout obtient le meilleur IoU pour les AR, mais l'Attention U-Net est retenu comme meilleur modèle global, car il améliore davantage la détection des TC, la classe la plus rare et la plus difficile à segmenter.

7 Conclusion et travail futur

Ce projet a exploré plusieurs stratégies pour améliorer la détection des TC et des AR à partir du jeu de données ClimatNet. Les méthodes d'échantillonnage par fichiers et par patches permettent de réduire significativement la taille des données tout en maintenant des performances proches de la baseline, mais elles n'améliorent pas clairement la détection des TC, qui restent la classe la plus difficile à segmenter.

Parmi les architectures testées, l'Attention U-Net se démarque comme le meilleur compromis global en améliorant l'IoU des TC de 0.3486 à 0.3765 par rapport à la baseline [1], tout en conservant des performances compétitives sur les AR. Les expériences avec le ConvLSTM démontrent que la dimension temporelle est une piste intéressante, mais limitée par le manque de continuité temporelle entre les snapshots de ClimateNet et par les contraintes mémoire du GPU. Le MC Dropout, quant à lui, apporte un gain de robustesse ainsi qu'une estimation de l'incertitude particulièrement utile dans ce contexte.

Dans l'ensemble, nos résultats confirment que le principal défi demeure le fort déséquilibre des classes. Avec seulement 0.46% des pixels TC, les modèles peinent à trouver le bon équilibre entre rappel et IoU. La détection de tous les TC sans déclencher trop de fausses alarmes reste donc un défi ouvert.

Pour les travaux futurs, trois pistes se révèlent prometteuses. Premièrement, l'utilisation de données temporellement continues comme ERA5 permettrait de mieux exploiter les architectures qui incorporent une dimension temporelle ainsi que le post-traitement. Deuxièmement, le jeu de données ClimateNet s'arrête en 2013. Il serait pertinent d'intégrer des données plus récentes afin de mieux capturer l'évolution du climat, surtout avec l'intensification des événements extrêmes observée depuis. Finalement, l'accès à des GPU avec plus de mémoire permettraient d'explorer des architectures plus expressives, des séquences temporelles plus longues et potentiellement des modèles de fondation tel que ClimaX [10] qui est pré-entraîné sur les données de ERA5.

8 Contributions

Les contributions individuelles des cinq membres de l'équipe sont les suivantes :

Rémi a reproduit le modèle de Kapp-Schwoerer et al. (2020) afin de l'utiliser comme point de référence (*baseline*) pour le reste du projet. Il a également exploré les pistes de travaux futurs proposées dans cet article.

Alexis a reproduit le meilleur modèle (expérience 7) de Lacombe et al. (2023). Il a aussi implémenté différents modèles hybrides avec des architectures ConvLSTM pour tenter de capturer la dimension temporelle des événements climatiques.

Laurent a reproduit l'expérience 5 de Lacombe et al. (2023). Ensuite, il a mis en place et évalué deux stratégies visant à reproduire les résultats du *baseline* en utilisant une quantité réduite des données d'entraînement.

Martin et **Étienne** ont développé et entraîné les architectures UNet standard et Attention UNet. Ils ont aussi créé le pipeline de post-traitement des prédictions et ont évalué les modèles sur une autre source de données.

9 Déclaration de l'utilisation des outils basés sur l'IA

Des outils basés sur l'IA ont été utilisés uniquement en soutien aux membres de l'équipe. Pour la rédaction, DeepL a servi à traduire de l'anglais vers le français, et QuillBot à corriger le texte. Claude et ChatGPT ont été consultés pour comprendre et clarifier des concepts/articles techniques. En ce qui concerne le code, Claude et ChatGPT ont été utilisés pour le débogage, notamment pour résoudre des problèmes associés à l'exécution d'expériences sur la grappe de calcul, puisque cet environnement était nouveau pour la plupart des membres de l'équipe. De plus, GitHub Copilot apparaît comme contributeur sur GitHub parce qu'il a été accidentellement activé pour réviser la première *pull request* sur le *data_downloader*. Le texte final du rapport et des livrables précédents a été rédigé par les cinq membres de l'équipe, et tout le code a été compris et validé par l'équipe.

References

[1] Kapp-Schwoerer et al. "Spatio-temporal segmentation and tracking of weather patterns with light-weight Neural Networks". In: *AI for Earth Sciences Workshop at NeurIPS* (2020).

[2] Chen et al. "Encoder-Decoder with Atrous Separable Convolution for Semantic Image Segmentation". In: *Proceedings of the European Conference on Computer Vision (ECCV)* (2018).

[3] Prabhat et al. "ClimateNet: an expert-labeled open dataset and deep learning architecture for enabling high-precision analyses of extreme weather". In: *Geoscientific Model Development Discussions* (2020).

[4] Wu et al. "CGNet: A Light-weight Context Guided Network for Semantic Segmentation". In: *IEEE Transactions on Image Processing* (2019).

[5] Hendren Lacombe Grossman and Lüdeke. *Improving extreme weather events detection with light-weight neural networks*. 2023. arXiv: `2304.00176 [cs.CV]`. URL: `https://arxiv.org/abs/2304.00176`.

[6] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. "U-Net: Convolutional Networks for Biomedical Image Segmentation". In: 2015. URL: `https://arxiv.org/abs/1505.04597`.

[7] Shruti Jadon. "A survey of loss functions for semantic segmentation". In: 2020. URL: `https://arxiv.org/abs/2006.14822`.

[8] Kaiming He et al. "Deep residual learning for image recognition". In: 2016. URL: `https://arxiv.org/abs/1512.03385`.

[9] Xingjian Shi et al. "Convolutional LSTM Network: A Machine Learning Approach for Precipitation Nowcasting". In: *CoRR* abs/1506.04214 (2015). arXiv: `1506.04214`. URL: `http://arxiv.org/abs/1506.04214`.

[10] Tung Nguyen et al. "ClimaX: A Foundation Model for Weather and Climate". In: *arXiv preprint arXiv:2301.10343* (2023). URL: `https://arxiv.org/abs/2301.10343`.

A Échantillonage

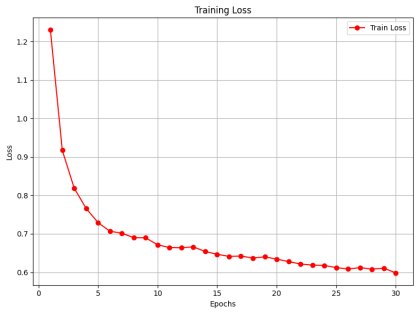
Table 7: Comparaison des expériences d'échantillonnage par patches. $\star$ configuration retenue. Les valeurs en **gras** indiquent le meilleur résultat par colonne.

#	Expérience	Cyclone tropical (TC)				Rivière atmosphérique (AR)			
		IoU	Préc.	Rap.	Spéc.	IoU	Préc.	Rap.	Spéc.
0	Reproduction baseline [1]	**0.3486**	0.4599	0.5903	0.9961	0.3848	0.4543	0.7157	0.9491
1	BG 10%	0.3142	0.4395	0.5244	0.9963	0.3829	0.4984	0.6231	0.9628
2	BG 15%	0.3005	0.5150	0.4191	**0.9978**	0.3821	0.5464	0.5597	0.9725
3	BG 20% $\star$	0.3126	0.4538	0.5012	0.9967	0.3854	0.5065	0.6172	0.9644
4	BG 25%	0.3025	0.3890	0.5764	0.9950	0.3850	0.4890	0.6442	0.9601
5	BG 30%	0.3061	0.4437	0.4968	0.9966	0.3891	0.4996	0.6377	0.9622
6	BG 40%	0.3105	0.4422	0.5104	0.9964	0.3680	0.4576	0.6529	0.9542
7	BG 20% + all features	0.3321	**0.5211**	0.4781	0.9976	**0.3893**	0.5242	0.6022	0.9676
8	BG 20% + TC $\times 6$	0.3071	0.4750	0.4650	0.9972	0.3764	**0.5513**	0.5427	**0.9738**
9	BG 20% + TC $\times 12$	0.2876	0.4023	0.5021	0.9959	0.3817	0.4995	0.6182	0.9633
10	BG 20% + CE pondérée	0.3185	0.4014	0.6068	0.9950	0.3834	0.5185	0.5955	0.9672
11	BG 20% + Jaccard pondérée	0.2469	0.2731	**0.7207**	0.9894	0.3588	0.4179	**0.7173**	0.9408

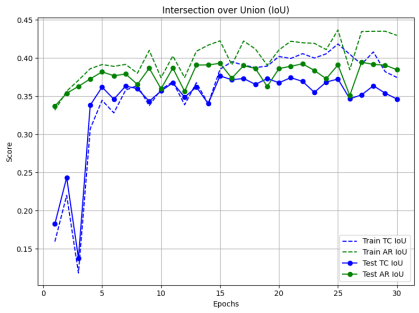
Table 8: Réduction de la taille des données selon la technique d'optimisation.

Technique	**Configuration**	**Train (GB)**		**Test (GB)**		**Total (GB)**		**Réduction (%)**
		Avant	Après	Avant	Après	Avant	Après	
1	Top 75% (4 carac.)	8.45	6.34	1.30	0.98	9.75	7.32	25.0
1	Top 64% (4 carac.)	8.45	5.41	1.30	0.83	9.75	6.24	36.0
2	Patches (4 carac.)	8.45	3.49	1.30	0.86	9.75	4.35	**55.4**
2	Patches (16 carac.)	25.37	15.30	3.89	4.50	29.26	19.80	32.3

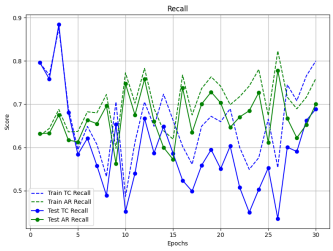
B Expériences U-Net



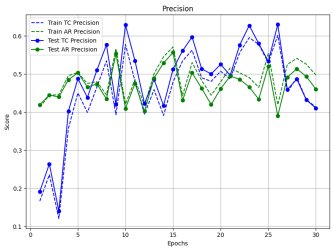
(a) Loss



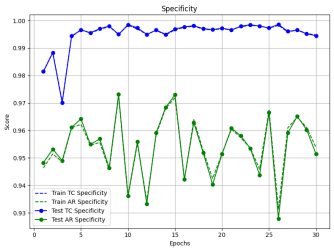
(b) Intersection over Union (IoU)



(c) Rappel (Recall)



(d) Précision



(e) Spécificité

Figure 3: Évolution des métriques d'entraînement et de test sur 30 époques.

C Expériences ConvLSTM

Table 9: Comparaison des expériences ConvLSTM sur l'ensemble de test (époque 15). Sauf indication contraire : perte d'entropie croisée pondérée $[0.5, 5.0, 2.0]$, caractéristiques TMQ, U850, V850, PSL [1], séquences consécutives, `features=[8, 16, 32, 64]` pour ConvLSTM-UNet, et `hidden_channels=[8, 16, 8]` pour ConvLSTM pur. T = longueur de séquence. †Caractéristiques : TMQ, WS850, VRT850, PSL, PRECT [5]. *OOM: erreur due au manque de mémoire. Les valeurs en **gras** indiquent le meilleur résultat par colonne.

#	Configuration	Cyclone tropical (TC)				Rivière atmosphérique (AR)			
		IoU	Préc.	Rap.	Spéc.	IoU	Préc.	Rap.	Spéc.
Référence									
0	Reproduction baseline [1]	0.3486	0.4599	0.5903	0.9961	0.3848	0.4543	0.7157	0.9491
ConvLSTM-UNet									
1	ConvLSTM-UNet, $T = 2$	0.3569	0.4998	0.5552	0.9969	0.3551	0.5965	0.4673	0.9813
2	ConvLSTM-UNet, $T = 3$	0.3495	0.4870	0.5531	0.9967	0.3786	0.5235	0.5777	0.9689
3	ConvLSTM-UNet, $T = 4$	0.2433	0.2587	**0.8034**	0.9869	0.3771	0.5928	0.5090	0.9794
4	#2 avec caractéristiques [5]†	0.3044	0.6330	0.3697	0.9988	0.3788	0.4755	0.6507	0.9575
5	#2 avec poids TC=7.0 dans la perte	**0.3699**	0.5381	0.5421	0.9974	0.3186	0.6452	0.3863	0.9874
6	#2 avec poids AR=3.0 dans la perte	0.2956	0.3616	0.6184	0.9939	0.3372	0.5847	0.4433	0.9814
7	#2 avec perte de Jaccard	0.2811	**0.6351**	0.3352	**0.9989**	0.3454	0.5921	0.4532	0.9815
8	#2 avec séquences du même jour seulement	0.2096	0.3313	0.3634	0.9957	0.3807	0.5019	0.6119	0.9609
9	#2 avec features=[16, 32, 64, 128]	0.3457	0.4863	0.5446	0.9968	0.3615	0.6041	0.4738	0.9816
10	#2 avec 16 caractéristiques	0.3384	0.5196	0.4926	0.9974	0.3808	0.5837	0.5227	0.9779
ConvLSTM-UNet-GC									
11	ConvLSTM-UNet-GC, $T = 3$	0.3381	0.4329	0.6069	0.9984	0.3831	0.5389	0.5698	0.9711
12	#11 avec ajout de trames *BG* pour continuité	0.1716	0.5704	0.1970	0.9992	0.2343	0.3913	0.3687	0.9661
13	#11 avec dropout = 0.1	0.2927	0.3330	0.7073	0.9920	0.3746	0.4699	0.6489	0.9567
14	#11 avec 16 caractéristiques	0.3527	0.5015	0.5431	0.9970	**0.3856**	0.4961	0.6338	0.9619
15	#14 avec poids [0.36, 30.0, 5.88]	0.2557	0.2768	0.7700	0.9887	0.3474	0.3999	**0.7260**	0.9355
16	#11 avec *CosineAnnealingLR*	0.3317	0.4214	0.6092	0.9953	0.3708	0.5414	0.5407	0.9729
17	#11 sur 30 époques	0.3327	0.4531	0.5558	0.9962	0.3450	0.5922	0.4525	0.9816
*Autres architectures limitées par *OOM*									
18	ConvLSTM-UNet (décodeur LSTM), $T = 2$	0.3377	0.4905	0.5201	0.9970	0.2500	**0.6916**	0.2814	**0.9926**
19	ConvLSTM pur, $T = 2$	0.1325	0.1941	0.2948	0.9932	0.2766	0.3972	0.4768	0.9571
20	#19 avec hidden_channels=[16, 32, 16]	0.1372	0.2038	0.2958	0.9936	0.2902	0.3556	0.6120	0.9343

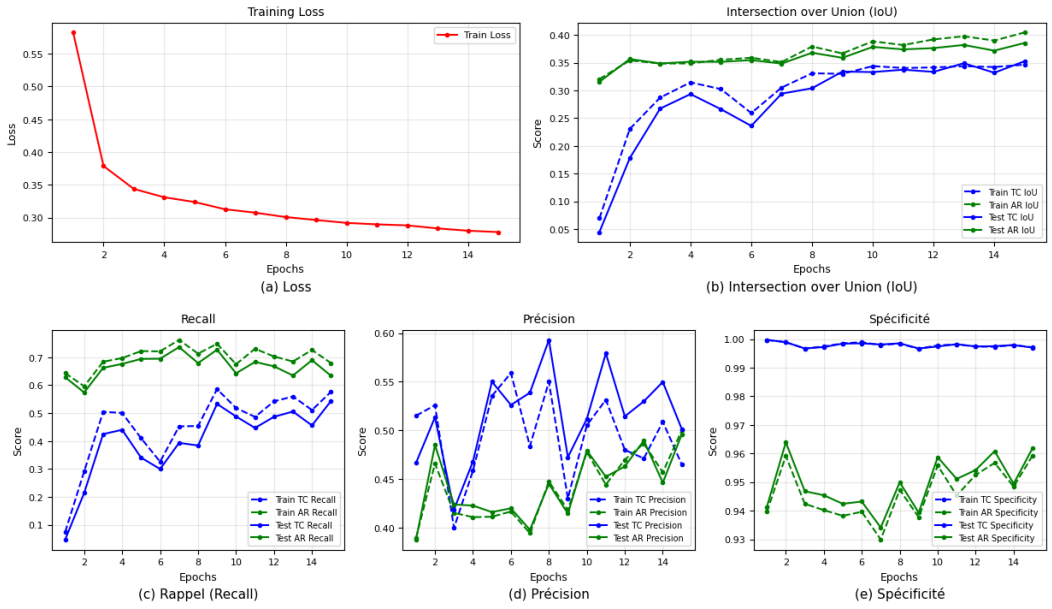


Figure 4: Évolution des métriques d'entraînement et de test pour l'expérience **14** des ConvLSTM.

D Exploitation du travail futur de l'aticle de 2020 de Kapp-Schwoerer et al.

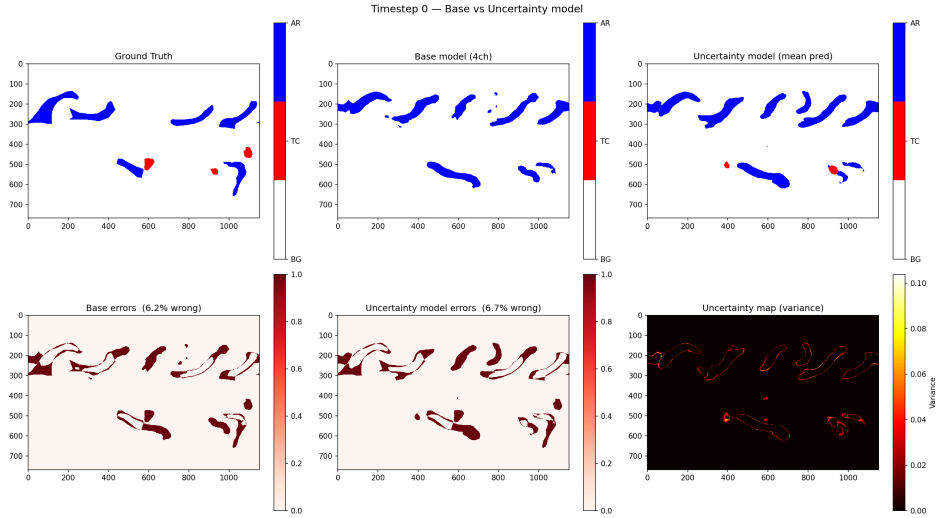


Figure 5: Visualisation de l'incertitude des predictions pour le modèle de base avec quantification d'incertitude.

E Post-traitement

#	Caractéristiques	Cyclone tropical (TC)				Rivière atmosphérique (AR)			
		IoU	Préc.	Rap.	Spéc.	IoU	Préc.	Rap.	Spéc.
1	4 Carac. de CGNet	0.3612	**0.5772**	0.4912	0.9967	**0.3873**	0.6310	**0.5008**	**0.9627**
2	8 Carac. : CGNet + Ingé.	**0.3646**	0.4753	**0.6100**	**0.9983**	0.3800	0.6599	0.4726	0.9564
3	8 Carac. : CGNet + Conservées	0.3611	0.5350	0.5263	0.9973	0.3798	0.6807	0.4622	0.9531
4	12 Carac. : CGNet + Ingé. + Conservées	0.3590	0.4740	0.5967	0.9982	0.3754	**0.6978**	0.4483	0.9491
5	Toutes les 20 carac.	0.3605	0.5749	0.4916	0.9967	0.3694	0.6510	0.4606	0.9548

Table 10: Comparaison des performances du Attention U-Net avec différents ensembles de caractéristiques sur l'ensemble de test. Tous les modèles ont été entraînés pour 25 époques. Les valeurs en **gras** indiquent le meilleur résultat par colonne.

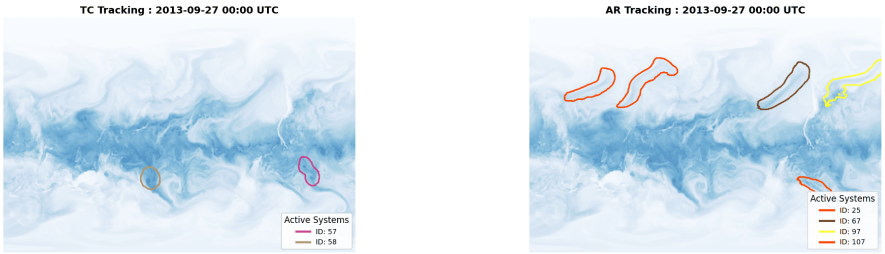


Figure 6: Post-traitement sur l'ensemble de données de test de ClimateNet. Le modèle utilisé est le AttentionUNet entraînés sur les 4 caractéristiques de CGNet pendant 25 epochs. Les images sont des liens vers des GIFs dans le dépôt GitHub.

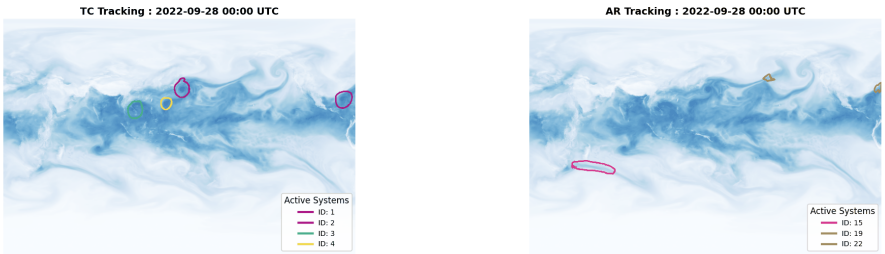


Figure 7: Post-traitement sur l'ensemble de données ERA5 du 26 au 29 septembre 2022. Le modèle utilisé est le AttentionUNet entraînés sur les 4 caractéristiques de CGNet pendant 25 epochs. Les images sont des liens vers des GIFs dans le dépôt GitHub.

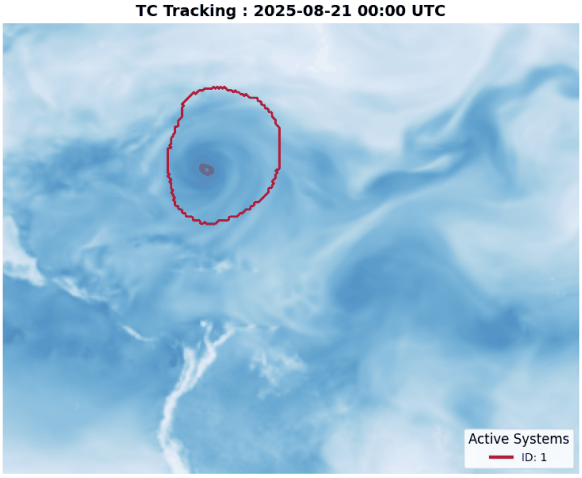


Figure 8: Post-traitement des cyclones tropicaux sur l'ensemble de données ERA5 du 19 au 22 août 2025. La figure est agrandi sur la zone des Caraïbes pour vieux voir comment le modèle suit l'ouragan Erin. Le modèle utilisé est le AttentionUNet entraînés sur les 4 caractéristiques de CGNet pendant 25 epochs. L'image est un lien vers un GIF dans le dépôt GitHub.